

UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN



**BASES DE DATOS
CONCURRENCIA Y RECUPERABILIDAD**

GUÍA DE EJERCICIOS

1 Serializabilidad / Recuperabilidad de Historias

- 1.1. Dada la siguiente historia (*schedule*) que es *conflicto-serializable* transformarla en una historia serial mediante una secuencia de intercambios no conflictivos de acciones adyacentes.

$$H_1 = r_1(A); w_1(A); r_2(A); w_2(A); r_1(B); w_1(B); r_2(B); w_2(B);$$

- 1.2. Dadas las siguientes historias (*schedules*):

$$H_1 = r_1(X); r_4(Y); w_1(X); w_4(X); r_3(X); c_1; w_4(Y); w_3(Y); w_4(Z); c_4; w_3(X); c_3.$$

$$H_2 = r_1(X); w_2(X); r_1(Y); w_1(X); w_1(Y); c_1; r_2(Z); w_2(Y); c_2.$$

$$H_3 = w_1(X); w_2(X); w_2(Y); c_2; w_1(Y); c_1; w_3(X); w_3(Y); c_3.$$

$$H_4 = w_2(X); r_1(X); r_2(Z); r_1(Y); w_2(Y); w_1(Y); c_2; w_1(X); c_1.$$

$$H_5 = r_1(X); r_4(Y); r_3(X); w_1(X); w_4(Y); w_3(X); c_1; w_3(Y); w_4(Z); c_3; W_4(X); c_4.$$

(a) Indicar para cada una si es NoRC, RC, ACA o ST.

(b) Indicar cuáles podrían producir un *dirty read* o un *lost update*.

- 1.3. Clasificar según recuperabilidad las siguientes historias:

$$H_1 = r_1(X); r_2(Z); r_1(Z); r_3(X); r_3(Y); w_1(X); c_1; w_3(Y); c_3; r_2(Y); w_2(Z); w_2(Y); c_2.$$

$$H_2 = r_1(X); r_2(Z); r_1(Z); r_3(X); r_3(Y); w_1(X); w_3(Y); r_2(Y); w_2(Z); w_2(Y); c_1; c_2; c_3.$$

$$H_3 = r_1(X); r_2(Z); r_3(X); r_1(Z); r_2(Y); r_3(Y); w_1(X); c_1; w_2(Z); w_3(Y); w_2(Y); c_3; c_2.$$

- 1.4. Dadas las siguientes transacciones:

$$T_1 = w_1(B); r_1(C); w(C);$$

$$T_2 = r_2(D); r_2(B); w_2(C);$$

(a) Dar una historia H_1 que no sea RC.

(b) Dar una historia H_2 que sea ACA pero no ST.

(c) Dar una historia H_3 que sea ST.

2 Introducción a Protocolos de Bloqueo o *Locking*

- 2.1. Dada las siguientes transacciones escritas como Read/Write:

$$T_1 = r_1(A); w_1(A); r_1(B); w_1(B)$$

$$T_2 = r_2(A); w_2(A); r_2(B); w_2(B)$$

Nota: Si bien ambas parecen iguales en realidad pueden realizar operaciones diferentes sobre los ítems leídos.

(a) Incorporar locks para que llevarlas a un modelo con *locking* binario

(b) Realizar una historia legal pero no serializable

- 2.2. Dadas las siguientes transacciones que deben ser ejecutadas en forma concurrente, indicar una historia legal en el cual se produzca un *dirty read*:

$$T_1 = l_1(A); A = A + 2; l_1(B); B = A + 5; u_1(B); u_1(A); l_1(C); C = 2 * C; u_1(C)$$

$$T_2 = l_2(A); A = A + 1; l_2(E); E = A + 8; l_2(D); u_2(E); D = D/5; u_2(A); u_2(D)$$

2.3. Considerando la siguiente historia con el modelo introductorio de LOCK/UNLOCK.

- (a) Determinar si es serializable.
- (b) En caso afirmativo, proponer un schedule serial equivalente

T_1	T_2	T_3	T_4
$l(A)$			
	$l(C)$		
$l(B)$			
	$u(C)$		
$u(A)$			
$u(B)$			
	$l(B)$		
	$u(B)$		
		$l(B)$	
		$u(B)$	
			$l(C)$
			$l(D)$
			$l(E)$
			$u(C)$
		$l(C)$	
			$u(D)$
		$u(C)$	
			$u(E)$

2.4. Dadas las transacciones del ejercicio 2.1 llevarlas a un modelo de locking binario tal que ambas sean 2PL.

2.5. Dadas las siguientes transacciones:

$$T_1 = l_1(A); l_1(B); u_1(A); u_1(B)$$

$$T_2 = l_2(B); l_2(A); u_2(B); u_2(A)$$

- (a) Responder: ¿Cumplen las transacciones T_1 y T_2 con 2PL?
- (b) Realizar un entrelazado tal que se produzca un *deadlock*

3 Sistemas de Bloqueo con varios modos

3.1. Dadas las siguientes transacciones T_1 y T_2 .

$$T_1 = wl_1(A); A = A + 1; wl_1(B); B = A + B; u_1(A); u_1(B)$$

$$T_2 = rl_2(A); wl_2(C); C = A + 1; wl_2(D); D = 1; u_2(A); u_2(D); u_2(C)$$

- (a) Construir un schedule legal serializable que no sea serial
- (b) Responder: ¿Cumplen las transacciones T_1 y T_2 con 2PL?

3.2. Dadas las transacciones T_1 , T_2 , T_3 y T_4 con el siguiente entrelazado:

T_1	T_2	T_3	T_4
	$rl(A)$		
		$rl(A)$	
	$wl(B)$		
	$u(A)$		
		$wl(A)$	
	$u(B)$		
$rl(B)$			
		$u(A)$	
			$rl(B)$
$rl(A)$			
			$u(B)$
$wl(C)$			
$u(A)$			
			$wl(A)$
			$u(A)$
$u(B)$			
$u(C)$			

- (a) Determinar si es serializable y, en caso afirmativo, proponer las correspondientes historias seriales equivalentes.

3.3. Dadas las transacciones T_1 , T_2 , T_3 , T_4 y T_5 con el siguiente entrelazado:

$H_1 : rl_1(A); rl_2(A); rl_3(A); u_1(A); rl_5(B); u_5(B); wl_1(B); wl_2(C); u_1(B); rl_3(B); u_3(A); u_2(A);$
 $wl_4(A); u_4(A); rl_1(D); u_1(D); wl_4(D); u_4(D); wl_5(A); u_5(A); u_2(C); u_3(B)$

- (a) Dibujar el entrelazado en forma tabular.
(b) Determinar si es serializable y, en caso afirmativo, proponer las correspondientes historias seriales equivalentes.
(c) Para cada transacción indicar si cumple con ser 2PL. Si no lo son modificarlas para que lo sean.
(d) Sobre el ítem anterior ubicar *commits* para que sean 2PL estricto y 2PL riguroso.

3.4. Dadas las transacciones:

$T_1 : rl_1(A); wl_1(B); u_1(A); u_1(B)$

$T_2 : rl_2(A); u_2(A); rl_2(B); u_2(B)$

$T_3 : wl_3(A); u_3(A); wl_3(B); u_3(B)$

$T_4 : rl_4(B); u_4(B); wl_4(A); u_4(A)$

- (a) Armar una historia legal y serializable y dar la correspondiente planificación serial.
(b) Dibujar el Grafo de Precedencia de la siguiente historia H_1 ($SG(H_1)$) y determinar si es serializable:
 $H_1 : wl_3(A); rl_4(B); u_3(A); rl_1(A); u_4(B); wl_3(B); rl_2(A); u_3(B); wl_1(B); u_2(A);$
 $u_1(A); wl_4(A); u_1(B); rl_2(B); u_4(A); u_2(B)$
(c) Reescribir las transacciones para que adhieran al protocolo 2PL.

3.5. Dadas las transacciones T_1 , T_2 , T_3 con el siguiente entrelazado H_1 :

$H_1 : wl_3(B); rl_3(B); rl_1(A); rl_3(C); u_3(B); rl_1(B); u_1(B); wl_2(B); u_3(C); rl_2(C);$
 $u_1(A); wl_2(A); u_2(B); u_2(A); rl_1(C); rl_3(D); u_3(D); u_1(C); wl_2(C); u_2(C)$

- (a) Indicar a qué historia serial es equivalente H_1 . Justificar la respuesta.
- (b) Responder: ¿Existe algún schedule legal no serializable que se pueda construir con T_1 , T_2 y T_3 ? Justificar la respuesta. En caso afirmativo, mostrar un ejemplo y justificar.
- (c) Responder: ¿Es posible identificar en H_1 un caso en el que pueda producirse un *dirty read*? Justificar la respuesta. En caso afirmativo, mostrar un ejemplo y justificar

Nota: Dibujar el entrelazado en forma tabular para poder visualizar mejor la historia.

- 3.6.** Dada la siguiente historia H_1 , sobre el conjunto de transacciones T_1 , T_2 , T_3 , T_4 y el conjunto de items X, Y :

$H_1 = rl_3(X); rl_2(X); wl_3(Y); u_3(X); wl_2(X); u_3(Y); rl_4(Y); u_2(X); rl_1(Y); rl_4(X); u_1(Y); u_4(X); wl_1(X); u_4(Y); u_1(X); c_3; c_2; c_4; c_1$

Nota: Asuma que entre un wl y su correspondiente ul ocurre solamente una escritura del ítem (no ocurre una lectura)

- (a) Responder: ¿ H_1 es Legal? ¿Todas la T_i son 2PL? .
- (b) Hacer el $SG(H_1)$ e indicar si es serializable. En caso afirmativo obtener todas las historias seriales equivalentes.
- (c) Clasificar H_1 con respecto a recuperabilidad: ST, ACA, RC, (no RC). Justificar
- (d) Idem anterior pero cambiando el orden de los commits por: $c_4; c_1; c_2; c_3$
- (e) Ubicar los commits y modificar H_1 para que las transacciones sean 2PL Estrictas

- 3.7.** Dadas las siguientes transacciones:

$T_1 = rl_1(B); u_1(B); wl_1(A); u_1(A)$

$T_2 = rl_2(A); wl_2(A); u_2(A)$

$T_3 = rl_3(A); rl_3(B); u_3(A); u_3(B)$

Nota: Asumir que entre un wl y su correspondiente ul ocurre solamente una escritura del ítem (no ocurre una lectura)

- (a) Se pide hallar una historia H tal que sea: Legal; No Serial; Serializable; Equivalente a la ejecución serial T_3, T_2, T_1 y ACA.
- (b) Hacer el $SG(H)$ para verificar la seriabilidad del H hallado. Justificar las respuestas con respecto a legalidad y recuperabilidad.
- (c) Transformar las transacciones a 2PL Estricto y 2PL Riguroso

- 3.8.** Dadas las siguientes transacciones:

$T_1 = rl_1(A); r_1(A); rl_1(B); r_1(B); wl_1(B); w_1(B); u_1(A); u_1(B)$

$T_2 = rl_2(A); r_2(A); rl_2(B); r_2(B); u_2(A); u_2(B)$

- (a) Realizar un entrelazado serializable no serial
- (b) Proponer un entrelazado donde un *upgrade lock* sea denegado y deba esperar.

- 3.9.** Tomando las siguientes transacciones

$T_1 = rl_1(A); r_1(A); wl_1(A); w_1(A); u_1(A)$

$T_2 = rl_2(A); r_2(A); wl_2(A); w_2(A); u_2(A)$

- (a) Proponer un entrelazamiento que de un *deadlock* como resultado.

- (b) Reescribir las transacciones, pero ahora, usando un modelo que soporte *update lock* y verificar si puede resolverse el *deadlock* anterior.

3.10. Dadas las siguientes historias

$$H_1 = r_1(A); r_2(B); r_3(C); r_1(B); r_2(C); r_3(D); w_1(C); w_2(D); w_3(E)$$

$$H_2 = r_1(A); r_2(B); r_3(C); r_1(B); r_2(C); r_3(D); w_1(A); w_2(B); w_3(C)$$

$$H_3 = r_1(A); r_2(B); r_3(C); r_1(B); r_2(C); r_3(A); w_1(A); w_2(B); w_3(C)$$

Se pide:

- Insertar locks de lectura y escritura de tal forma que **no haya** *upgrade locks*. Explicar que ocurriría en la ejecución con un planificador/scheduler que soporte locks de lectura y de escritura
- Insertar locks de lectura y escritura de tal forma que **si haya** *upgrade locks* y describir que ocurre con la ejecución.
- Insertar ahora locks de lectura, escritura y de actualización (*update locks*). Describir que ocurre en la ejecución.

3.11. Dadas las siguientes transacciones

$$T_1 = l_1(A); r_1(A); l_1(B); w_1(B); u_1(A); u_1(B)$$

$$T_2 = l_2(C); r_2(C); l_2(A); w_2(A); u_2(C); u_2(A)$$

$$T_3 = l_3(B); r_3(B); l_3(C); w_3(C); u_3(B); u_3(C)$$

$$T_4 = l_4(D); r_4(D); l_4(A); w_4(A); u_4(D); u_4(A)$$

Se pide:

- Transformar las mismas a un modelo de locks ternario
- Realizar un entrelazado tal que se produzca un *deadlock* y todas las transacciones queden esperando
- Realizar el grafo *wait-for*
- Describir que ocurre en el esquema anterior si se utiliza prevención de *deadlocks* con
 - Un esquema Wait-Die
 - Un esquema Wound-Wait

3.12. Para cada una de las historias siguientes:

$$H_1 = r_1(A); r_2(B); w_1(C); r_3(D); r_4(E); w_3(B); w_2(C); w_4(A); w_1(D);$$

$$H_2 = r_1(A); r_2(B); r_3(C); w_1(B); w_2(C); w_3(D);$$

$$H_3 = r_1(A); r_2(B); r_3(C); w_1(B); w_2(C); w_3(A);$$

Se pide:

- Ubicar un *read lock* inmediatamente antes de cada acción de lectura, un *write lock* inmediatamente antes de cada acción de escritura y *unlocks* de modo que las transacciones cumplan con 2PL
- En base a lo anterior decir que acciones son denegadas y cuando ocurre un *deadlock*
- Dibujar la evolución del grafo *wait-for*
- Asumiendo un orden T_1, T_2, T_3, T_4 y un mecanismo de detección de *deadlock* explicar que ocurre si este mecanismo usa:
 - Un esquema Wait-Die
 - Un esquema Wound-Wait

3.13. Dado el siguiente *schedule* s en el modelo de locking ternario:

$$s = wl_1(A); rl_1(B); u_1(B); wl_3(C); wl_2(B); rl_3(D); u_3(C); rl_2(C);$$

$$wl_3(D); u_2(B); u_3(D); u_1(A); u_2(C); c_3; c_1; c_2$$

Responda **verdadero o falso**:

- a) _____ s es 2PL.
- b) _____ s soporta update lock.
- c) _____ s soporta upgrading lock.
- d) _____ $T_1 \rightarrow T_3 \rightarrow T_2$ es la única historia serial equivalente al entrelazado anterior.
- e) _____ s es RC (recuperable).
- f) _____ s es ACA.

4 Métodos con *timestamping* y Multiversión

4.1. Detallar que ocurre en las siguientes secuencias, el orden es el orden temporal en el que ocurren los eventos si:

- $H_1 = st_1; st_2; r_1(A); r_2(B); w_2(A); w_1(B)$
- $H_2 = st_1; r_1(A); st_2; r_2(B); r_2(A); w_1(B)$
- $H_3 = st_1; st_2; st_3; r_1(A); r_2(B); w_1(C); r_3(B); r_3(C); w_2(B); w_3(B)$

- (a) El planificador usa *timestamping* y no es multiversión.
- (b) El planificador usa multiversión y protocolo **MVTO**

4.2. Dada la siguiente historia en un planificador con timestamp:

$$H_1 = st_1; st_2; r_2(X); st_3; st_4; r_1(Y); r_4(Z); w_3(X); w_3(Y); w_4(Z); w_2(X); w_1(Y); r_3(Z)$$

en donde los valores iniciales de $X; Y; Z$ son 0 y pasa que:

T_1 escribe $Y = 1$
 T_2 escribe $X = 2$
 T_3 escribe $X = 3; Y = 30$
 T_4 escribe $Z = 4$

- (a) Decir que pasa en cada acción y que valores quedan en $X; Y; Z$ si el planificador **no usa multiversión**
- (b) Decir que pasa en cada acción y que valores se tendrían para $X; Y; Z$ si el planificador **usa multiversión MVTO**

4.3. Dadas las siguientes transacciones:

$$T_1 = r_1(A); r_1(B); w_1(X); w_1(B)$$

$$T_2 = r_2(C); r_2(B); r_2(A); w_2(A); w_2(B)$$

$$T_3 = r_3(B); r_3(X); w_3(A); w_3(B)$$

Se pide:

- (a) Realizar una historia para un planificador basado en timestamping multiversión **MVTO** que produzca un comportamiento físicamente irrealizable.

4.4. Sean T_1, T_2 y T_3 , transacciones que operan sobre los mismos ítems A, B y C de una base de datos. Las siguientes secuencias muestran el orden de ejecución de las transacciones T_1, T_2 y T_3 , .

- (a) Asignar los timestamps e insertar los comienzos de transacción para que con timestamping monoversión sea posible realizar la secuencia sin que se produzca rollback.

Secuencias a analizar:

- $r_1(A); r_2(A); w_1(A); r_3(A); w_3(A); w_2(A)$
- $r_1(A); r_3(C); r_2(C); w_3(A); r_2(B); r_3(B); w_2(B); w_2(A)$

4.5. Dadas las siguientes historias:

$H_1 = st_2; r_2(Z); r_2(Y); w_2(Y); st_3; r_3(Y); r_3(Z); st_1; r_1(X); w_1(X); w_3(Y); w_3(Z);$
 $r_2(X); r_1(Y); w_1(Y); w_1(X)$
 $H_2 = st_3; r_3(Y); r_3(Z); st_1; r_1(X); w_1(X); w_3(Y); w_3(Z); st_2; r_2(Z); r_1(Y); w_1(Y);$
 $r_2(Y); w_2(Y); r_2(X); w_2(X)$ Se pide:

- (a) Verificar según un planificador basado en timestamp si las historias son permitidas o no.
- (b) Modificar una de las historias para adaptarla a un planificador multiversión **MV2PL** que demore alguna de las transacciones en **un paso final**.
- (c) Modificar una de las historias para que un planificador multiversión **MVTO** tenga que hacer rollback.

4.6. Dadas las siguientes transacciones:

$T_1 = r_1(A); r_1(B); w_1(A)$
 $T_2 = r_2(B); r_2(C); w_2(A)$
 $T_3 = r_3(C); r_3(B); w_3(B)$

Se pide:

- (a) Realizar una historia para un scheduler basado en *timestamp* **sin multiversión** tal que el orden inicio sea T_1, T_3, T_2 y que realice un rollback de T_3 por un comportamiento físicamente irrealizable *write too late*.

Nota: No olvidarse de indicar los comienzos de las transacciones.

4.7. Dada la siguiente historia de un planificado con timestamp **MVTO** (Multiversion Time Ordering):

$H = st_1; st_2; r_2(X); st_3; st_4; r_1(Y); r_4(Z); w_3(X); w_3(Y); w_4(Z); w_2(X); w_1(Y); r_3(Z)$

Suponer que t_1 escribe $Y = 10$, t_2 escribe $X = 8$, t_3 escribe $X = 2$ e $Y = 4$, t_4 escribe $Z = 6$

Asumir que si una transacción finaliza exitosamente, realiza commit inmediatamente después de la última operación. Responder **verdadero o falso**

- a) _____ Al final de la ejecución de H , $X = 2$, $Y = 4$ y $Z = 6$.
- b) _____ T_3 aborta por *read too late*.
- c) _____ Al final de la ejecución de H_1 , $X = 8$, $Y = 10$ y $Z = 6$.
- d) _____ T_1 y T_2 abortan por *write too late*.
- e) _____ T_1 y T_2 realizan *commit* antes que T_3

5 Recuperabilidad

5.1. Dada la siguiente historia:

$H = r_1(Z); w_1(U); c_1; w_2(X); w_2(Y); r_3(U); w_3(X); w_2(Z); c_2; r_3(Y); r_4(Z); w_3(Y); c_3; r_4(U); w_4(U); c_4$

Suponer que inicialmente todos los ítems tienen valor 0 y cada operación WRITE incrementa en 1 el valor anterior. Escribir la secuencia de los registros de *log* correspondientes a la ejecución exitosa de H, según:

- (a) *Undo logging* con *checkpointing quiescente* (incluir un *checkpoint* en el *log*)
- (b) *Redo logging* con *checkpointing no quiescente* (incluir un *checkpoint* en el *log* tal que ocurra durante la ejecución de H)
- (c) *Undo/Redo logging* sin *checkpointing*.

5.2. Dada la siguiente secuencia de registros en el *undo-log*, escritos por el *log manager*, correspondientes a la ejecución de tres transacciones T_1, T_2 y T_{12} , sobre los ítems A, B, C, D, E y R:

$\langle START T_1 \rangle; \langle T_1, A, 10 \rangle; \langle START T_2 \rangle; \langle T_2, B, 20 \rangle; \langle START T_{12} \rangle$
 $\langle T_1, C, 30 \rangle; \langle T_2, D, 40 \rangle; \langle COMMIT T_2 \rangle; \langle T_{12}, R, 12 \rangle; \langle T_1, E, 50 \rangle;$
 $\langle ABORT T_{12} \rangle; \langle COMMIT T_1 \rangle$

Describir las acciones que debería realizar el *recovery manager* (incluyendo los cambios tanto en los ítems en disco como en el *log*) si ocurre un *crash* y el último registro de *log* en disco es:

- (a) $\langle START T_2 \rangle$
- (b) $\langle COMMIT T_2 \rangle$
- (c) $\langle T_1, E, 50 \rangle$
- (d) $\langle COMMIT T_1 \rangle$

5.3. Dada la siguiente secuencia de registros en el *undo-log* generados por el *log manager* correspondientes a la ejecución de las transacciones T_1, T_2, T_3 y T_4 :

$\langle START T_1 \rangle; \langle T_1, A, 8 \rangle; \langle START T_2 \rangle; \langle START T_3 \rangle; \langle T_2, B, 16 \rangle;$
 $\langle START T_4 \rangle; \langle T_4, E, 24 \rangle; \langle T_2, D, 32 \rangle; \langle T_4, K, 9 \rangle; \langle T_4, F, 40 \rangle;$
 $\langle ABORT T_4 \rangle; \langle COMMIT T_3 \rangle; \langle T_2, G, 48 \rangle; \langle COMMIT T_2 \rangle; \langle T_1, C, 56 \rangle;$
 $\langle COMMIT T_1 \rangle$

- (a) Describir las acciones que debería realizar el *recovery manager* (incluyendo los cambios tanto en los ítems en disco como en el *log*) si ocurre un *crash* y el último registro de *log* en disco es:
 - i. $\langle START T_4 \rangle$
 - ii. $\langle T_1, C, 56 \rangle$
- (b) Ídem ejercicio anterior pero asumiendo *redo logging* y que el último registro de *log* en disco es:
 - i. $\langle COMMIT T_3 \rangle$
 - ii. $\langle COMMIT T_1 \rangle$

5.4. Dada la siguiente secuencia de registros *undo/redo-log* generados por el *log manager* correspondientes a la ejecución de cuatro transacciones T_1, T_2, T_3 y T_4 :

$\langle START T_1 \rangle; \langle T_1, A, 100, 110 \rangle; \langle START T_2 \rangle; \langle T_2, B, 200, 210 \rangle;$
 $\langle START T_3 \rangle; \langle T_1, C, 300, 310 \rangle; \langle T_3, D, 400, 410 \rangle; \langle T_2, E, 40, 41 \rangle;$

$\langle T_3, F, 500, 510 \rangle$; $\langle COMMIT T_3 \rangle$; $\langle COMMIT T_2 \rangle$; $\langle START T_4 \rangle$;
 $\langle T_1, G, 600, 610 \rangle$; $\langle T_4, H, 700, 710 \rangle$; $\langle COMMIT T_1 \rangle$; $\langle COMMIT T_4 \rangle$

Describe las acciones que debería realizar el recovery manager -incluyendo los cambios tanto en los ítems en disco como en el log- si ocurre un crash y el último registro de log en disco es:

- (a) $\langle START T_3 \rangle$
- (b) $\langle COMMIT T_3 \rangle$
- (c) $\langle T_1, G, 600, 610 \rangle$
- (d) $\langle COMMIT T_4 \rangle$

5.5. Dada la siguiente secuencia de registros de *undo-log*:

$\langle START T_1 \rangle$; $\langle T_1, A, 60 \rangle$; $\langle COMMIT T_1 \rangle$; $\langle START T_2 \rangle$; $\langle T_2, A, 10 \rangle$;
 $\langle START T_3 \rangle$; $\langle T_3, B, 20 \rangle$; $\langle T_2, C, 30 \rangle$; $\langle START T_4 \rangle$; $\langle T_3, D, 40 \rangle$;
 $\langle T_4, F, 70 \rangle$; $\langle COMMIT T_3 \rangle$; $\langle T_2, E, 50 \rangle$; $\langle COMMIT T_2 \rangle$; $\langle T_4, B, 80 \rangle$;
 $\langle COMMIT T_4 \rangle$

Suponer que se inicia *checkpoint no-quiesscente* inmediatamente después de que cada uno de los siguientes registros de *log* han sido escritos en el *pool* de *buffers* de memoria:

- 1) $\langle T_1, A, 60 \rangle$
- 2) $\langle T_2, A, 10 \rangle$
- 3) $\langle T_3, B, 20 \rangle$
- 4) $\langle T_3, D, 40 \rangle$
- 5) $\langle T_2, E, 50 \rangle$

Para cada uno indicar:

- (a) Cuándo el registro $\langle END CKPT \rangle$ es escrito.
- (b) Para cada punto donde fuera posible que ocurriera un *crash*, hasta dónde se debería examinar el *log* para hallar todas las posibles transacciones incompletas.

5.6. Dados los mismos datos del ejercicio anterior pero ahora asumiendo *redo logging* indicar para cada punto de 1) a 5):

- (a) En qué punto el registro $\langle END CKPT \rangle$ podría ser escrito.
- (b) Para cada punto donde fuera posible que ocurriera un *crash*, hasta dónde se debería examinar el *log* para hallar todas las posibles transacciones completas. Considere los dos casos posibles
 - i. que el registro $\langle END CKPT \rangle$ **fué** escrito antes del *crash*.
 - ii. que el registro $\langle END CKPT \rangle$ **no fué** escrito antes del *crash*.

- 5.7. Se tiene el siguiente log de un planificador que utiliza estrategia REDO Logging con checkpoints no-quiescente. Asumiendo que hay un crash luego del paso 27 marcar con **verdadero o falso**.

1	< START T1 >
2	< T1, A, 5 >
3	< START T2 >
4	< T1, W, 90 >
5	< T1, R, 15 >
6	< T1, A, 8 >
7	< COMMIT T1 >
8	< T2, B, 10 >
9	< START T3 >
10	< T3, M, 7 >
11	< T2, C, 14 >
12	< START T4 >
13	< T4, H, 2 >
14	< ABORT T3 >
15	< START CKPT (T2,T4) >
16	< T2, D, 15 >
17	< T4, I, 20 >
18	< START T5 >
19	< START T6 >
20	< T6, W, 1 >
21	< T5, O, 1 >
22	< T5, P, 3 >
23	< END CKPT >
24	< COMMIT T2 >
25	< T5, Q, 5 >
26	< COMMIT T4 >
27	< COMMIT T6 >

Asumiendo que hay un crash luego del paso 27 marcar con **verdadero o falso**.

- Se tienen que rehacer T2, T4, T5 y T6, ya que se garantiza que aquellas transacciones que comitearon antes del Start Ckpt estan en disco
- Se rehacen solamente las transacciones que comenzaron luego del Start Ckpt. Todo lo anterior ya se encuentra en disco
- Despues del paso 27 todas las transacciones commitearon, con lo cual solo hay que hacer flush del log en disco
- Se rehacen desde el Start Ckpt todas las transacciones incompletas y ademas se rehacen T5 y T6

Asumiendo que hay un crash luego del paso 20 marcar con **verdadero o falso**.

- Se tienen que rehacer T1, T2, T3 y T4
- Se tienen que rehacer unicamente T1

- 5.8. Dada la siguiente secuencia de registros *undo/redo-log* con *nonquiescent checkpointing* generados por el *log manager* correspondientes a la ejecución de las transacciones T_1 a T_5 :

- < START T_1 >
- < T_1 , A, 26, 33 >
- < START T_2 >
- < START T_3 >
- < T_2 , C, 51, 34 >
- < T_3 , E, 21, 35 >
- < T_1 , B, 25, 24 >
- < T_1 , A, 33, 20 >
- < T_2 , D, 18, 36 >
- < COMMIT T_1 >
- < START T_4 >
- < T_4 , F, 19, 37 >
- < T_2 , D, 36, 4 >
- < START CKPT (T_2, T_3, T_4) >

15. $\langle T_2, C, 34, 10 \rangle$
16. $\langle T_4, F, 37, 41 \rangle$
17. $\langle \text{START } T_5 \rangle$
18. $\langle T_3, E, 35, 52 \rangle$
19. $\langle T_5, G, 27, 43 \rangle$
20. $\langle \text{COMMIT } T_2 \rangle$
21. $\langle \text{END CKPT} \rangle$
22. $\langle T_5, G, 43, 28 \rangle$
23. $\langle T_4, H, 26, 23 \rangle$
24. $\langle \text{COMMIT } T_5 \rangle$
25. $\langle \text{COMMIT } T_4 \rangle$

Asuma que las entradas en el log son de la forma $\langle \text{transaccion}, \text{item}, \text{valor anterior}, \text{valor nuevo} \rangle$.
Para los siguientes posibles escenarios de fallas:

- (a) El sistema *crashea* justo antes de que la línea 23 sea escrita en el disco.
- (b) Idem anterior pero para la línea 24.
- (c) Idem anterior pero para la línea 25.
- (d) El sistema *crashea* inmediatamente después de que la línea 25 fuera escrita en el disco.

Completar una tabla con los valores de los items A, B, C, D, E, F, G y H en el disco después de una recuperación exitosa.

6 Notación

En esta sección veremos un resumen de la notación utilizada para escribir las transacciones y las historias.

H_i	Historia. Historia o <i>schedule i</i> .
T_i	Transacción. Forma de notar a la transacción <i>i</i> .
$r_i(A)$	Lectura. La transacción <i>i</i> lee el ítem A.
$w_i(A)$	Escritura. La transacción <i>i</i> escribe el ítem A.
c_i	Commit. La transacción <i>i</i> realiza un <i>commit</i> .
a_i	Abort. Se aborta la transacción <i>i</i> .
$l_i(A)$	Lock. La transacción <i>i</i> realiza un <i>bloqueo o lock</i> sobre el ítem A. Usado en lockeo binario.
$u_i(A)$	UnLock. La transacción <i>i</i> libera los <i>bloqueos o locks</i> previos sobre el ítem A. Usado en todos los modelos, se asume que libera todos los <i>locks</i> tomados.
$rl_i(A)$	Lock de lectura o compartido. La transacción <i>i</i> realiza un <i>bloqueo o lock</i> de lectura sobre el ítem A.
$wl_i(A)$	Lock de escritura o exclusivo. La transacción <i>i</i> realiza un <i>bloqueo o lock</i> exclusivo o de escritura sobre el ítem A.
$ul_i(A)$	Lock de actualización. La transacción <i>i</i> realiza un <i>bloqueo o lock</i> de actualización (<i>update lock</i>) sobre el ítem A.
st_i	Comienzo de transacción. La transacción <i>i</i> comienza y se le asigna un <i>timestamp</i> . Usado en planificadores con <i>timestamp</i> .
$R_i(A, B, \dots)$	Comienzo y Conjunto de lectura. La transacción <i>i</i> comienza con el conjunto de lectura (A,B...). Usado para planificadores basados en validación.
$W_i(A, B, \dots)$	Finalización y Conjunto de escritura. La transacción <i>i</i> finaliza con el conjunto de escritura (A,B...). Usado para planificadores basados en validación.
V_i	Validación. La transacción <i>i</i> es validada. Usado para planificadores basados en validación.
$TS(T)$	Timestamp indica el timestamp de la transacción <i>T</i>